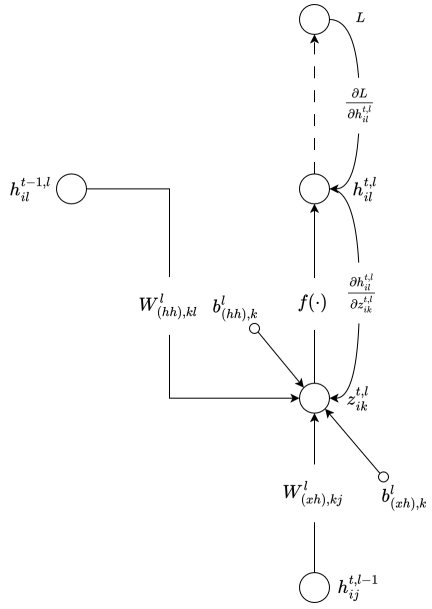
Neural_Networks

Task	Convolutional	Transformer
Dense Prediction (segmentation & depth estimation)	FCN, U-Net, monoDepth	DPT, SETR
Image Recognition	ResNet-50, EfficientNet	ViT, DinoV2
Object Detection	YOLO, Faster R-CNN, CenterNet, FCOS	DETR
Image Captioning	CNN+LSTM,	CLIP
Keypoint detection (local feature matching)	Superpoint, superglue	Loftr

	Low dimensional states	High-dimensional states
Static dataset	Classic supervised learning (ML)	Deep supervised learning (DL)
Agent/environment interaction	Tabular reinforcement learning	Deep reinforcement learning

RNN

1. RNN Cell (CMU 11-785)



1.
2. An RNN Cell forward
    1. takes as input: x or h_prev_l, h_prev_t
    2. produces: h_next and cache: (h_next, h_prev_l, h_prev_t, dh_next) (for backprop, in most implementations)
3. An RNN Cell backward
    1. takes as input, upstream gradient dh_next, h_next, h_prev_l, h_prev_t
    2. produces: dx or dh_prev_l, dh_prev_t
4. RNN Cell:

```
import numpy as np
from .activation import *

class RNNCell:

    def __init__(self, input_size, hidden_size):

        self.input_size = input_size
        self.hidden_size = hidden_size

        # Activation function for
        self.activation = Tanh()

        # hidden dimension and input dimension
        h = self.hidden_size
        d = self.input_size

        # Weights and biases
        self.W_ih = np.random.randn(h, d)
        self.W_hh = np.random.randn(h, h)
```

```
        self.b_ih = np.random.randn(h)
        self.b_hh = np.random.randn(h)

        self.dW_ih = np.zeros((h, d))
        self.dW_hh = np.zeros((h, h))
        self.db_ih = np.zeros(h)
        self.db_hh = np.zeros(h)

    def init_weights(self, W_ih, W_hh, b_ih, b_hh):
        self.W_ih = W_ih
        self.W_hh = W_hh
        self.b_ih = b_ih
        self.b_hh = b_hh

    def zero_grad(self):
        h, d = self.hidden_size, self.input_size
        self.dW_ih = np.zeros((h, d))
        self.dW_hh = np.zeros((h, h))
        self.db_ih = np.zeros(h)
        self.db_hh = np.zeros(h)

    def __call__(self, x, h_prev_t):
        return self.forward(x, h_prev_t)

    def forward(self, x, h_prev_t):
        h_next =  h_prev_t @ self.W_hh.T + self.b_ih + x @ self.W_ih.T + self.b_hh
        out = self.activation.forward(h_next)
        return out

    def backward(self, dh_next, h_next, h_prev_l, h_prev_t):
        dz = self.activation.backward(dh_next, h_next)
        batch_size = dh_next.shape[0]
        # 1) Compute the averaged gradients of the weights and biases
        self.dW_ih += dz.T @ h_prev_l / batch_size
        self.dW_hh += dz.T @ h_prev_t / batch_size
        self.db_ih += np.sum(dz.T, axis=1) / batch_size
        self.db_hh += np.sum(dz.T, axis=1) / batch_size

        # 2) Compute dx, dh_prev_t
        dx = dz @ self.W_ih
        dh_prev_t = dz @ self.W_hh

        return dx, dh_prev_t
```

2. RNN Classifier (CMU 11-785)
    1. Example: many-to-one, multiple layers RNN Classifier
    2. RNN Classifier forward,
    3. RNN Classifier backward,
        1. An important principle for BPTT: very simple rule: for each node with multiple descendants in the computational graph, simply add up the delta vectors coming from all of the descendants.
    4. Code:

```
import numpy as np

from HW3.P1.mytorch.rnn_cell import *
from HW3.P1.mytorch.linear import *


class RNNPhonemeClassifier(object):
    """RNN Phoneme Classifier class."""

    def __init__(self, input_size, hidden_size, output_size, num_layers=2):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers

        # TODO: Understand then uncomment this code :)
        self.rnn = [
            (
                RNNCell(input_size, hidden_size)
                if i == 0
                else RNNCell(hidden_size, hidden_size)
            )
            for i in range(num_layers)
        ]
        self.output_layer = Linear(hidden_size, output_size)

        # store hidden states at each time step, [(seq_len+1) * (num_layers, batch_size, hidden_size)]
        self.hiddens = []

    def init_weights(self, rnn_weights, linear_weights):
        """Initialize weights.

        Parameters
        ----------
        rnn_weights:
                    [
```

```
                    [W_ih_l0, W_hh_l0, b_ih_l0, b_hh_l0],
                    [W_ih_l1, W_hh_l1, b_ih_l1, b_hh_l1],
                    ...
                ]

    linear_weights:
                    [W, b]

    """
    for i, rnn_cell in enumerate(self.rnn):
        rnn_cell.init_weights(*rnn_weights[i])
    self.output_layer.W = linear_weights[0]
    self.output_layer.b = linear_weights[1].reshape(-1, 1)

def __call__(self, x, h_0=None):
    return self.forward(x, h_0)

def forward(self, x, h_0=None):
    """RNN forward, multiple layers, multiple time steps.
    x: input sequence: (N, T, D)
    self.x: cache (for backprop) of input sequence: (N, T, D)
    h0: initial hidden state (optional), zeros if not given: (L, N, H)
    self.hiddens: cache (for backprop) of computed hidden states (h_next): (T+1, L, N, H)
    hiddens: computed hidden states for all the layers (L, N, H)
    h_next: hidden state (N, H)
    logits: output (N, H_out)
    """
    N, T, D = x.shape
    L = self.num_layers
    H = self.hidden_size

    if h_0 is None:
        hiddens = np.zeros((L, N, H), dtype=np.float64)
    else:
        L, _, H = h_0.shape
        assert L == self.num_layers, H == self.hidden_size
        hiddens = h_0
    self.hiddens.append(hiddens.copy())
    self.x = x
    for t in range(T):
        for l in range(L):
            # Compute h_prev_l
            if l == 0:
                h_prev_l = x[:, t, :]
            else:
                h_prev_l = hiddens[l - 1, :, :]
            # Compute h_prev_t
            h_prev_t = hiddens[l]
            # Perform forward on RNN Cell
            h_next = self.rnn[l].forward(h_prev_l, h_prev_t)
            # Update hiddens for next time step
            hiddens[l] = h_next

        self.hiddens.append(hiddens.copy())

    logits = self.output_layer.forward(h_next)
    return logits

def backward(self, delta):
    """RNN Back Propagation Through Time (BPTT).
    N: number of samples or batch size
    L: number of layers
    H: hidden dimension
    D: input dimension
    H_out: output dimension
    delta: Upstream gradient, dL/dy from linear layer before loss function
    i.e. in last RNN Cell, h_next -> y -> L: (N, H_out)
    self.hiddens: Contains h_next computed in each RNN Cell: (T+1, L, N, H)
    self.x: input sequence: (N, T, D)
    dh_next: contains upstream gradients: (L, N, H)
    for backprop, cache = (h_next, h_prev_l, h_prev_t, dh_next)
    """
    T = self.x.shape[1]
    L = self.num_layers
    H = self.hidden_size
    N, _ = delta.shape
    dh_next = np.zeros((L, N, H), dtype=np.float64)
    # Compute upstream gradient, dL/dh_next, for last RNN Cell
    dh_next[-1] = self.output_layer.backward(delta)
    # self.output_layer.backward(delta)
    for t in reversed(range(T)):
        for l in reversed(range(L)):
            # Compute variables in cache need for backprop
            h_next = self.hiddens[t + 1][l, :, :]
            h_prev_t = self.hiddens[t][l, :, :]
            # h_prev_l can come from either input sequence or h_next from previous cell, l-1
            if l == 0:
                h_prev_l = self.x[:, t, :]
            else:
                h_prev_l = self.hiddens[t + 1][l - 1, :, :]
```

```
                # Done computing variables in cache needed for backprop
                # Perform backprop on RNN Cell
                h_prev_l, h_prev_t = self.rnn[l].backward(
                    dh_next[l], h_next, h_prev_l, h_prev_t
                )
                # Update dh_next for the next previous time step
                dh_next[l] = h_prev_t
                # Add downstream gradient to the upstream gradient of the next previous cell step
                if l != 0:
                    dh_next[l - 1] += h_prev_l
        return dh_next / N


if __name__ == "__main__":
    input_size = 3
    hidden_size = 4
    output_size = 5
    rnn = RNNPhonemeClassifier(input_size, hidden_size, output_size)

    N = 10
    seq_len = 3
    x = np.random.random((N, seq_len, input_size))
    logits = rnn.forward(x)
    # print(logits.shape)
    # print(logits)
    delta = np.random.random((N, output_size))
    dh = rnn.backward(delta)
    print(dh)
    print(dh.shape)
```

	shape
W_{xh}	(D, H)
W_{hh}	(H, H)
x_t	(N, D)
h	(N, H)
b_h	(H,)
z_t	(N, H)

3. RNN Cell (CS231n)
    1. forward
        1. $U = x_t \cdot W_{xh}$
        2. $V = h_{t-1} \cdot W_{hh}$
        3. $z_t = U + V + b_h$
        4. $h_t = \tanh(z_t)$
        5. code:

```
def rnn_step_forward(x, prev_h, Wx, Wh, b):
    """
    Run the forward pass for a single timestep of a vanilla RNN that uses a tanh
    activation function.

    The input data has dimension D, the hidden state has dimension H, and we use
    a minibatch size of N.

    Inputs:
    - x: Input data for this timestep, of shape (N, D).
    - prev_h: Hidden state from previous timestep, of shape (N, H)
    - Wx: Weight matrix for input-to-hidden connections, of shape (D, H)
    - Wh: Weight matrix for hidden-to-hidden connections, of shape (H, H)
    - b: Biases of shape (H,)

    Returns a tuple of:
    - next_h: Next hidden state, of shape (N, H)
    - cache: Tuple of values needed for the backward pass.
    """
    next_h, cache = None, None
    ##############################################################################
    # TODO: Implement a single forward step for the vanilla RNN. Store the next  #
    # hidden state and any values you need for the backward pass in the next_h   #
    # and cache variables respectively.                                          #
    ##############################################################################
    z_t = x @ Wx + prev_h @ Wh + b
    next_h = np.tanh(z_t)
    # cache = (z_t, x, prev_h, Wx, Wh)
    cache = (next_h, x, prev_h, Wx, Wh)
    ##############################################################################
    #                               END OF YOUR CODE                             #
```

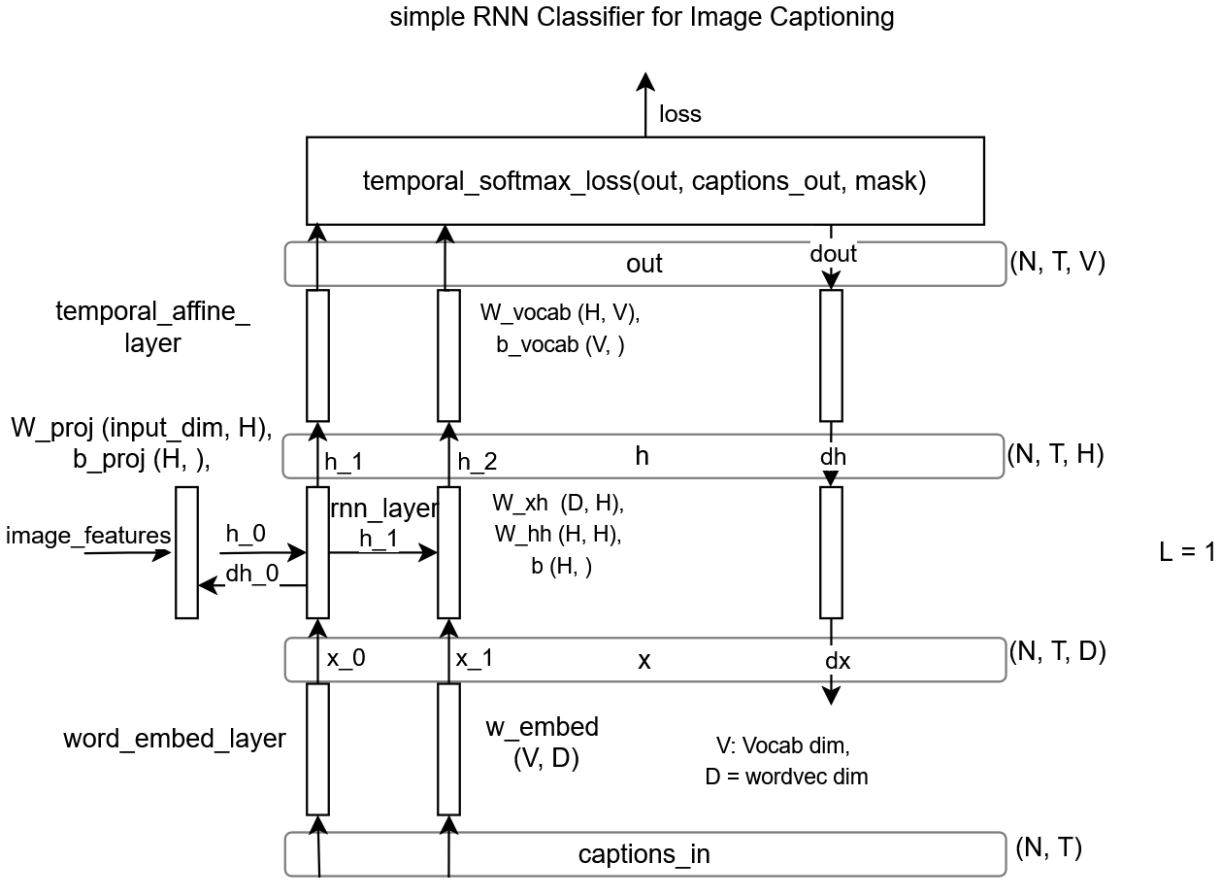
```
    ##############################################################################
    return next_h, cache
```

2. backward
    1. $\frac{\partial L}{\partial z_t} = \frac{\partial L}{\partial h_t} \odot (1 - \tanh^2(z_t))$
    2. $\frac{\partial L}{\partial U} = \frac{\partial L}{\partial z_t}$
    3. $\frac{\partial L}{\partial V} = \frac{\partial L}{\partial z_t}$
    4. $\frac{\partial L}{\partial W_{xh}} = (\frac{\partial L}{\partial z_t}^T \cdot x_t)^T$
    5. $\frac{\partial L}{\partial W_{hh}} = (\frac{\partial L}{\partial z_t}^T \cdot h_{t-1})^T$
    6. $\frac{\partial L}{\partial x_t} = \frac{\partial L}{\partial z_t} W_{xh}^T$
    7. $\frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial z_t} W_{hh}^T$
    8. $\frac{\partial L}{\partial b} = (\frac{\partial L}{\partial z_t}).sum(axis = 0)$
    9. Code:

```
def rnn_step_backward(dnext_h, cache):
    """
    Backward pass for a single timestep of a vanilla RNN.

    Inputs:
    - dnext_h: Gradient of loss with respect to next hidden state
    - cache: Cache object from the forward pass

    Returns a tuple of:
    - dx: Gradients of input data, of shape (N, D)
    - dprev_h: Gradients of previous hidden state, of shape (N, H)
    - dWx: Gradients of input-to-hidden weights, of shape (D, H)
    - dWh: Gradients of hidden-to-hidden weights, of shape (H, H)
    - db: Gradients of bias vector, of shape (H,)
    """
    dx, dprev_h, dWx, dWh, db = None, None, None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for a single step of a vanilla RNN.      #
    #                                                                            #
    # HINT: For the tanh function, you can compute the local derivative in terms #
    # of the output value from tanh.                                             #
    ##############################################################################
    # z_t, x_t, prev_h, Wx, Wh = cache
    next_h, x_t, prev_h, Wx, Wh = cache
    dz = dnext_h * (1 - next_h**2)
    dx = dz @ Wx.T
    dprev_h = dz @ Wh.T
    dWx = x_t.T @ dz
    dWh = prev_h.T @ dz
    db = np.sum(dz, axis=0)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return dx, dprev_h, dWx, dWh, db
```

4. RNN Classifier (CS231n)

1. many-to-many architecture for image captioning:



2. RNN Classifier forward,
3. RNN Classifier backward (BPTT),
4. Code:

```
def rnn_forward(x, h0, Wx, Wh, b):
    """
    Run a vanilla RNN forward on an entire sequence of data. We assume an input
    sequence composed of T vectors, each of dimension D. The RNN uses a hidden
    size of H, and we work over a minibatch containing N sequences. After running
    the RNN forward, we return the hidden states for all timesteps.

    Inputs:
    - x: Input data for the entire timeseries, of shape (N, T, D).
    - h0: Initial hidden state, of shape (N, H)
    - Wx: Weight matrix for input-to-hidden connections, of shape (D, H)
    - Wh: Weight matrix for hidden-to-hidden connections, of shape (H, H)
    - b: Biases of shape (H,)

    Returns a tuple of:
    - h: Hidden states for the entire timeseries, of shape (N, T, H).
    - cache: Values needed in the backward pass
    """
    h, cache = [], []
    ##############################################################################
    # TODO: Implement forward pass for a vanilla RNN running on a sequence of    #
    # input data. You should use the rnn_step_forward function that you defined  #
    # above. You can use a for loop to help compute the forward pass.            #
    ##############################################################################
    prev_h = h0
    _, T, _ = x.shape
    for t in range(T):
        next_h, next_cache = rnn_step_forward(
            x[:, t, :], prev_h,
            Wx, Wh, b
        )
        h.append(next_h)
        cache.append(next_cache)
        prev_h = next_h
    h = np.array(h)
    h = h.transpose(1, 0, 2)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return h, cache


def rnn_backward(dh, cache):
```

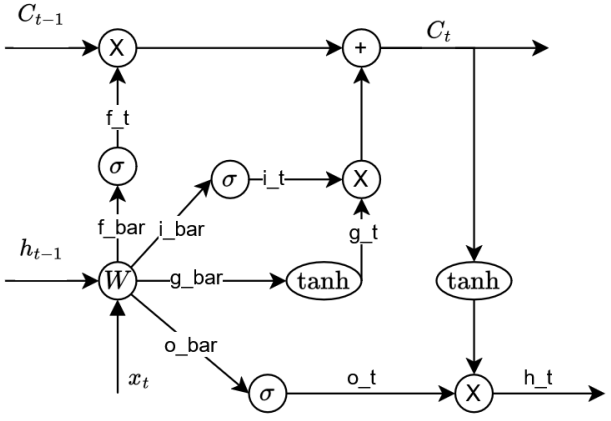
```
    """
    Compute the backward pass for a vanilla RNN over an entire sequence of data.

    Inputs:
    - dh: Upstream gradients of all hidden states, of shape (N, T, H)

    Returns a tuple of:
    - dx: Gradient of inputs, of shape (N, T, D)
    - dh0: Gradient of initial hidden state, of shape (N, H)
    - dWx: Gradient of input-to-hidden weights, of shape (D, H)
    - dWh: Gradient of hidden-to-hidden weights, of shape (H, H)
    - db: Gradient of biases, of shape (H,)
    """
    dx, dh0, dWx, dWh, db = None, None, None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for a vanilla RNN running an entire      #
    # sequence of data. You should use the rnn_step_backward function that you   #
    # defined above. You can use a for loop to help compute the backward pass.   #
    ##############################################################################
    N, T, H = dh.shape

    dx = []
    # Extract D from x.shape
    _, x, _, _, _ = cache[0]
    _, D = x.shape
    curr_dx = np.zeros((N, D))
    dh0 = np.zeros((N, H))
    dWx = np.zeros((D, H))
    dWh = np.zeros((H, H))
    db = np.zeros(H)
    for t in reversed(range(T)):
        # Compute upstream gradients as input to RNN cell that computes local derivatives,
        # and produces dx, dprev_h, dWh, dWx and db
        if t == T - 1:
            dnext_h = dh[:, t, :]
        else:
            dnext_h = dh[:, t, :] + dprev_h
        curr_dx, dprev_h, prev_dWx, prev_dWh, prev_db = rnn_step_backward(dnext_h, cache[t])
        dWx += prev_dWx
        dWh += prev_dWh
        db += prev_db
        dx.append(curr_dx)
    dh0 = dprev_h
    # Because the way its elements are appended, resort the order based on ascending index of time
    dx.reverse()
    dx = np.array(dx)
    dx = dx.transpose(1, 0, 2)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return dx, dh0, dWx, dWh, db
```

LSTM Cell

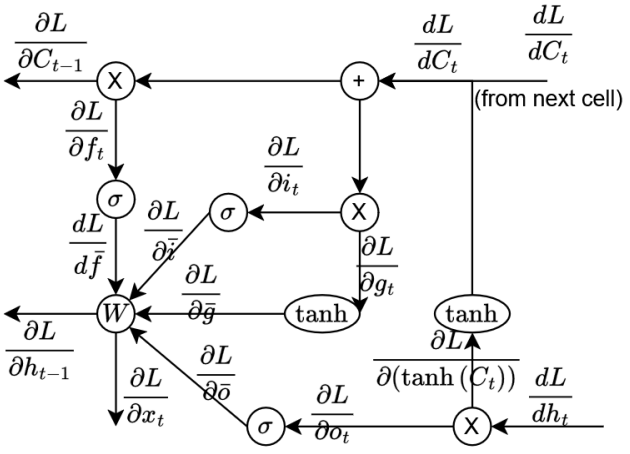
1. $\mathbf{i}_t = \sigma(\mathbf{W}_{ix}\mathbf{x}_t + \mathbf{W}_{ih}\mathbf{h}_{t-1})$,
2. $\mathbf{f}_t = \sigma(\mathbf{W}_{fx}\mathbf{x}_t + \mathbf{W}_{fh}\mathbf{h}_{t-1})$,
3. $\mathbf{o}_t = \sigma(\mathbf{W}_{ox}\mathbf{x}_t + \mathbf{W}_{oh}\mathbf{h}_{t-1})$,
4. $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_{cx}\mathbf{x}_t + \mathbf{W}_{ch}\mathbf{h}_{t-1})$,
5. $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$,

LSTM Cell

Forward



Backward



	shape
W_{xh}	(D, H)
W_{hh}	(H, H)
h_{t-1}	(N, H)
x_t	(N, D)
W	(D+H, 4H)
X	(N, H+D)
b	(4H,)
z	(N, 4H)

6. Forward
    1. $i_t = \sigma([x_t, h_{t-1}] \begin{bmatrix} W_{xi} \\ W_{hi} \end{bmatrix})$, input gate
    2. $f_t = \sigma([x_t, h_{t-1}] \begin{bmatrix} W_{xf} \\ W_{hf} \end{bmatrix})$, forget gate
    3. $o_t = \sigma([x_t, h_{t-1}] \begin{bmatrix} W_{xo} \\ W_{ho} \end{bmatrix})$, output gate
    4. $g_t = \tanh([x_t, h_{t-1}] \begin{bmatrix} W_{xg} \\ W_{hg} \end{bmatrix})$, 'gate' gate
    5. $\begin{pmatrix} \bar{i} \\ \bar{f} \\ \bar{o} \\ \bar{g} \end{pmatrix} = [x_t \quad h_{t-1}] \begin{bmatrix} W_{xi} & W_{xf} & W_{xo} & W_{xg} \\ W_{hi} & W_{hf} & W_{ho} & W_{hg} \end{bmatrix} + b$, shape: (n, 4h), where

       $z = \begin{pmatrix} \bar{i} \\ \bar{f} \\ \bar{o} \\ \bar{g} \end{pmatrix}, \quad X = [x_t \quad h_{t-1}], \quad W = \begin{bmatrix} W_{xi} & W_{xf} & W_{xo} & W_{xg} \\ W_{hi} & W_{hf} & W_{ho} & W_{hg} \end{bmatrix}$
    6. $i_t = \sigma(\bar{i})$
    7. $f_t = \sigma(\bar{f})$
    8. $o_t = \sigma(\bar{o})$
    9. $g_t = \tanh(\bar{g})$
    10. $C_t = f_t \odot C_{t-1} + i_t \odot g_t$
    11. $h_t = o_t \odot \tanh(C_t)$
    12. Code:

```python
def lstm_step_forward(x, prev_h, prev_c, Wx, Wh, b):
    """
    Forward pass for a single timestep of an LSTM.

    The input data has dimension D, the hidden state has dimension H, and we use
    a minibatch size of N.

    Inputs:
    - x: Input data, of shape (N, D)
    - prev_h: Previous hidden state, of shape (N, H)
    - prev_c: previous cell state, of shape (N, H)
    - Wx: Input-to-hidden weights, of shape (D, 4H)
    - Wh: Hidden-to-hidden weights, of shape (H, 4H)
    - b: Biases, of shape (4H,)

    Returns a tuple of:
    - next_h: Next hidden state, of shape (N, H)
    - next_c: Next cell state, of shape (N, H)
    - cache: Tuple of values needed for backward pass.
    """
    next_h, next_c, cache = None, None, None
    #############################################################################
    # TODO: Implement the forward pass for a single timestep of an LSTM.        #
    # You may want to use the numerically stable sigmoid implementation above.  #
    #############################################################################
    x_stack = np.hstack((x, prev_h))
    w_stack = np.vstack((Wx, Wh))
    z = x_stack @ w_stack + b  # shape: (N, 4*H)
    i_bar, f_bar, o_bar, g_bar = np.split(z, 4, axis=1)
    i_t = sigmoid(i_bar); f_t = sigmoid(f_bar); o_t = sigmoid(o_bar); g_t = np.tanh(g_bar)
    next_c = f_t * prev_c + i_t * g_t
    next_h = o_t * np.tanh(next_c)
    cache = (prev_h, x, prev_c, next_c,
```

```
            i_bar, f_bar, o_bar, g_bar,
            i_t, f_t, o_t, g_t,
            Wx, Wh)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return next_h, next_c, cache
```

7. Backward
    1. cache (to store in forward): $C_t, \bar{g}, \bar{o}, \bar{f}, \bar{i}, x_t, h_{t-1}, C_{t-1}, g_t, o_t, f_t, i_t, W_x, W_h$.
    2. $A \cdot B$ denotes matrix multiplication, $A \odot B$ denotes element-wise multiplication, $\sigma(x) = \text{sigmoid(x)} = \frac{1}{1+e^{-x}}$.
    3. $\frac{dL}{do_t} = \frac{\partial L}{\partial h_t}\frac{\partial h_t}{\partial o_t} = \frac{\partial L}{\partial h_t} \odot \tanh(C_t)$
    4. $\frac{\partial L}{\partial(\tanh C_t)} = \frac{\partial L}{\partial h_t}\frac{\partial h_t}{\partial(\tanh C_t)} = \frac{\partial L}{\partial h_t} \odot o_t$
    5. $\frac{\partial L}{\partial C_t} += \frac{\partial L}{\partial(\tanh(C_t))}\frac{\partial(\tanh(C_t))}{\partial C_t} = \frac{\partial L}{\partial(\tanh C_t)} \odot (1 - \tanh^2 C_t)$
    6. $\frac{\partial L}{\partial f_t} = \frac{\partial L}{\partial C_t} \odot C_{t-1}$
    7. $\frac{\partial L}{\partial C_{t-1}} = \frac{\partial L}{\partial C_t} \odot f_t$
    8. $\frac{\partial L}{\partial i_t} = \frac{\partial L}{\partial C_t} \odot g_t$
    9. $\frac{\partial L}{\partial g_t} = \frac{\partial L}{\partial C_t} \odot i_t$
    10. $\frac{\partial L}{\partial \bar{g}} = \frac{\partial L}{\partial g_t} \odot (1 - \tanh^2 \bar{g})$
    11. $\frac{\partial L}{\partial \bar{o}} = \frac{\partial L}{\partial o_t}\frac{\partial \sigma(\bar{o})}{\partial \bar{o}} = \frac{\partial L}{\partial o_t} \odot \sigma(\bar{o}) \odot (1 - \sigma(\bar{o}))$
    12. $\frac{\partial L}{\partial \bar{f}} = \frac{\partial L}{\partial f_t}\frac{\partial \sigma(\bar{f})}{\partial \bar{f}} = \frac{\partial L}{\partial f_t} \odot \sigma(\bar{f}) \odot (1 - \sigma(\bar{f}))$
    13. $\frac{\partial L}{\partial \bar{i}} = \frac{\partial L}{\partial i_t}\frac{\partial \sigma(\bar{i})}{\partial \bar{i}} = \frac{\partial L}{\partial i_t} \odot \sigma(\bar{i}) \odot (1 - \sigma(\bar{i}))$
    14. $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial z} \cdot W^T$
    15. $\frac{\partial L}{\partial W} = (\frac{\partial L}{\partial z}^T \cdot X)^T$
    16. $\frac{\partial L}{\partial b} = (\frac{\partial L}{\partial z}).sum(axis = 0)$
    17. Code:

```
def lstm_step_backward(dnext_h, dnext_c, cache):
    """
    Backward pass for a single timestep of an LSTM.

    Inputs:
    - dnext_h: Gradients of next hidden state, of shape (N, H)
    - dnext_c: Gradients of next cell state, of shape (N, H)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient of input data, of shape (N, D)
    - dprev_h: Gradient of previous hidden state, of shape (N, H)
    - dprev_c: Gradient of previous cell state, of shape (N, H)
    - dWx: Gradient of input-to-hidden weights, of shape (D, 4H)
    - dWh: Gradient of hidden-to-hidden weights, of shape (H, 4H)
    - db: Gradient of biases, of shape (4H,)
    """
    dx, dh, dc, dWx, dWh, db = None, None, None, None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for a single timestep of an LSTM.       #
    #                                                                            #
    # HINT: For sigmoid and tanh you can compute local derivatives in terms of  #
    # the output value from the nonlinearity.                                    #
    ##############################################################################
    prev_h, x, prev_c, next_c, i_bar, f_bar, o_bar, g_bar, i_t, f_t, o_t, g_t, Wx, Wh = cache
    do_t = dnext_h * np.tanh(next_c)
    dnext_c += dnext_h * o_t * (1 - np.tanh(next_c)**2)
    df_t = dnext_c * prev_c
    dprev_c = dnext_c * f_t
    di_t = dnext_c * g_t
    dg_t = dnext_c * i_t
    dg_bar = dg_t * (1 - np.tanh(g_bar)**2)
    do_bar = do_t * (1 - sigmoid(o_bar)) * sigmoid(o_bar)
    df_bar = df_t * (1 - sigmoid(f_bar)) * sigmoid(f_bar)
    di_bar = di_t * (1 - sigmoid(i_bar)) * sigmoid(i_bar)
    dz = np.hstack((di_bar, df_bar, do_bar, dg_bar))
    W = np.vstack((Wx, Wh))
    dx_ = dz @ W.T
    _, D = x.shape
    # _, H = prev_h.shape
    dx = dx_[:, :D]
    dprev_h = dx_[:, D:]
    X = np.hstack((x, prev_h))
    dW = (dz.T @ X).T
    dWx = dW[:D, :]
    dWh = dW[D:, :]
    db = dz.sum(axis=0)
    ##############################################################################
    #                               END OF YOUR CODE                             #
```

```
    ##############################################################################

    return dx, dprev_h, dprev_c, dWx, dWh, db
```

Flattening

1. Flattening dimensions is often used when we are doing matrix multiplications such as when preparing input to MLP.
2. During flattening the dimensions are 'aggregated' into one dimension, for example (3, 4, 5) into (3, 20).
3. Code:

```
# torch.flatten

x = torch.rand(3, 4, 5)
a = x.flatten(1, 2)
b = x.flatten()
print("x.shape", x.shape)
print("a.shape", a.shape)
print("b.shape", b.shape)
```

4. Example:
    1. mfccs shape: (T, 28)
    2. during training using MLP:
        1. x: frames: (N, T, D)
        2. N: batch_size
        3. T: time_window: $2 * \text{context\_window} + 1$
        4. D: 28 (dimension)
        5. reshape x, (N, $T * D$)->mlp->phonemes (n, 42)

Incorporating information in neural nets

1. Most of the times in neural nets we need to incorporate information using either summing or concatenation.
2. Based on information theory, as long as the information is there it should be usable. From information standpoint it does not really matter either we use summing or concatenation.
   2. Sum: less memory requirement
   3. Concatenate: more memory

KL divergence

1. KL divergence: take the expectation of the logarithm of the ratio of p to q as if you were to assume that x is sampled through p.
2. $KL(P||Q) = \sum_{i=1}^{n} p_i \log\left(\frac{p_i}{q_i}\right)$.

Diffusion models

1. Connection to VAEs
    1. Diffusion models can be considered as special case of hierarchical VAEs.
    2. However, in diffusion models:
        1. The encoder is fixed.
        2. The latent variables have the same dimension as the data.
        3. The denoising model is shared across different timestep.
        4. The model is trained with some reweighting of the variational bound.
2. DDPM is an image-generation model
    1. Generating images.
    2. Sampling images from a simple prior.
    3. Modeling the distrib of the data X of interest.
    4. Computing the prob of data p(x), x is an image.
    5. In contrast to discriminative models that models p(y|x).
    6. Training: Diffusion to get the input.
        1. Sample an image x from the training set.
        2. Sample noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$.
        3. Sample a variance $\beta \in (0, 1)$.
        4. Get the input $x_t = \sqrt{1-\beta}x + \sqrt{\beta}\epsilon$.

    5. reparametrization trick: $x = mean + std\_dev * Z$ to sample from normal distribution. Z is drawn from a standard normal distribution.
    6. Note: reparametrization trick is a useful mental step to do backprop through a sampling process. While the gradient of an individual sample indeed does not exist or isn't meaningful, it does exist in expectation, and the expectation is all you need to backprop. Papers that refer this as a trick may come from the years 2012-2014.
7. Training: Denoising to get the output
    1. Feed the input into the network.
    2. Network: U-Net f_θ.
    3. Predict the noise $f_\theta(x_t, t)$.
    4. Loss is MSE.
    5. input to model: $x_t = \sqrt{1-\beta}x + \sqrt{\beta}\epsilon$
    6. output: $\tilde{\epsilon}$, $\tilde{x} = \frac{x_t - \sqrt{\beta}\tilde{\epsilon}}{\sqrt{1-\beta}}$.
8. Inference: Iterative denoising

References:

1. LSTM: A detailed Explanation, Towards Data Science, Rian Dolphin.
2. Unsupervised Representation Learning by Predicting Image Rotations. Gidaris, et. al. Ecole des Ponts ParisTech.